

DevOps Assignment – Django + React

Project Overview

This project demonstrates containerization, CI/CD automation, and production-style deployment for a full-stack application using industry-standard DevOps practices.

Application: Hello World Full-Stack App

Backend: Django (REST API)

Frontend: React (Vite + TypeScript)

Repository: <https://github.com/Nexgensis/devops-assessment.git>

1. Prerequisites

Ensure the following tools are installed on your local machine:

- Git
 - Docker Desktop
 - Docker Compose
 - VS Code
 - GitHub account
 - Docker Hub account
-

2. Local Setup Guide (Step-by-Step)

2.1 Clone the Repository

```
git clone https://github.com/Nexgensis/devops-assessment.git
cd devops-assessment
```

2.2 Backend Dockerization (Django)

- Multi-stage Docker build used to reduce image size
- Application runs as a **non-root user** for security

Build image:

```
docker build -t <dockerhub-username>/django-backend ./backend
```

2.3 Frontend Dockerization (React)

- Multi-stage build: build stage + nginx serve stage
- Non-root user execution

Build image:

```
docker build -t <dockerhub-username>/react-frontend ./frontend
```

2.4 Docker Compose (Local Orchestration)

Run both services together:

```
docker-compose up -d
```

Access application:

- Frontend: http://localhost
 - Backend API: http://localhost:8000
-

3. Environment Configuration

Environment variables are managed using Docker Compose:

- `DEBUG=False`
- `API_URL=http://backend:8000`

No secrets or credentials are hardcoded.

4. CI/CD Pipeline (GitHub Actions)

Workflow Features

- Triggered on every push to `main` branch
- Builds Docker images
- Pushes images to Docker Hub automatically

Workflow location:

```
.github/workflows/docker-build.yml
```

Secrets configured in GitHub:

- DOCKERHUB_USERNAME
- DOCKERHUB_TOKEN

5. Production-Style Local Deployment

Since cloud billing was unavailable, a production-equivalent local deployment was implemented.

Steps:

```
docker pull <dockerhub-username>/django-backend
docker pull <dockerhub-username>/react-frontend
docker-compose up -d
```

Restart policies ensure resilience.

6. Screenshots

Screenshots included:

- Application running in browser
- Docker containers running (`docker ps`)
- Docker Desktop container view

7. Troubleshooting Log

Issue: Docker Desktop unable to start containers

Problem: Docker failed to start due to WSL2 backend issues on Windows.

Solution: - Restarted WSL using `wsl --shutdown` - Restarted Docker Desktop - Verified Docker engine before running containers

Learning: System-level dependencies must be validated before container orchestration.

8. Security & Best Practices Followed

- Multi-stage Docker builds
- Non-root containers
- `.dockerignore` usage
- No hardcoded secrets
- Environment-based configuration

9. Conclusion

This project demonstrates end-to-end DevOps practices including containerization, CI/CD automation, and production-style deployment, fulfilling all mandatory and evaluation criteria.

Author: ABEX